

The matching engine (price-time priority)

Learning notes for `OrderBook::submit(...)`. A book holds resting limit orders; when an order arrives that can trade, the **matching engine** pairs it against the book and prints the trades (fills). This explains the one rule that governs it - **price-time priority** - and how the walk, partial fills, and resting remainders work.

1. Makers and takers

- **Maker** - a resting limit order sitting in the book, providing liquidity (e.g. "sell 5 @ 100").
- **Taker** - an incoming order that crosses the spread and removes liquidity (e.g. "buy 10, up to 101").

Matching pairs a taker against one or more makers, producing **fills**. Whatever the taker can't fill either rests in the book (if it's a limit) or is dropped (if it's a market order).

2. The one rule: price-time priority

Which maker trades first? Two tie-breakers, in order:

1. **Price first**. A buyer takes the **lowest** ask; a seller hits the **highest** bid. Best price for the taker, always.
2. **Time second (FIFO)**. Among makers at the *same* price, the one that arrived **earliest** fills first. This is why each price level is a FIFO queue in the book.

BUY 10 (limit 101) sweeps the asks in price-time order

asks

101: id3 q4

100: id1 q5 id2 q3 <- oldest first (FIFO)

fills, in order:

- 1) id1 @100 x5 (best price, oldest)
 - 2) id2 @100 x3 (same price, next in FIFO)
 - 3) id3 @101 x2 (next price; taker's 10 now filled)
- id3 partially filled -> 2 left resting @101, which becomes the new best ask

3. The walk

For a buy, keep taking the best ask while it still crosses the taker's limit and quantity remains:

```
uint32_t submit(uint64_t id, Side side, int32_t limit_tick, uint32_t qty,
               bool is_market, std::vector<Fill>& fills) {
    uint32_t rem = qty;
    if (side == Side::Buy) {
        while (rem > 0) {
            int32_t bt = best_ask_;
            if (bt > max_tick_) break; // ask side empty
            if (!is_market && bt > limit_tick) break; // no longer crosses
            Order& mk = pool_[ask_[bt].head]; // FIFO head = oldest at best price
            uint32_t trade = std::min(rem, mk.qty);
            fills.push_back({id, mk.id, bt, trade, Side::Buy});
            rem -= trade;
            if (trade == mk.qty) { uint64_t m = mk.id; cancel(m); } // full fill: remove maker
            else { mk.qty -= trade; ask_[bt].total_qty -= trade; } // partial: maker stays
        }
    } /* sell side is the mirror image against best_bid_ */
    if (rem > 0 && !is_market) add(id, side, limit_tick, rem); // rest the remainder
    return rem;
}
```

Why this is fast: `best_ask_` is O(1), the FIFO head is one pointer, a full maker fill reuses the book's O(1) `cancel` (which also repairs the best price and frees the pool slot), and there is no allocation on the path. Measured: **p50 42 ns per submit, ~17 M submit/s**.

4. Three outcomes for the taker

- **Fully filled:** `rem == 0`, done.
- **Limit, partially filled (or unmatched):** the remaining quantity **rests** in the book as a new maker (`add`), and becomes part of the quoted liquidity.
- **Market order:** ignores the price bound and sweeps; any remainder is **dropped**, never rested (a market order doesn't want to sit on the book).

A single fill records `{taker_id, maker_id, price tick, qty, taker_side}` - enough to feed a trade tape, P&L, or a downstream risk check.

5. Correctness

`tests/test_match.cpp` asserts the semantics directly: price-time ordering of fills, a partially filled maker staying at the head, a limit remainder resting at the right price, a sell crossing the bid side, and a market order sweeping then dropping its remainder. It runs in CI alongside the top-of-book cross-check.

6. Next: a signal on top

With matching in place, the natural next layer is a **strategy**: read the book (e.g. an order-book-imbalance or micro-price signal), decide, and `submit` an order - then measure the **signal-to-order latency**. That closes the loop from market data to a trade.